

Discovering Token-Level Importance Metrics for Selective KV Cache Re-computation in Large Language Models

Aleksandr Golomazov
HSE University
avgolomazov@edu.hse.ru

Abstract

Large Language Models (LLMs) combined with Retrieval-Augmented Generation (RAG) systems often suffer from high Time-To-First-Token (TTFT) due to expensive prefill computation over long retrieved contexts. One promising approach to reduce this overhead is to store pre-calculated Key-Value (KV) caches for frequently reused documents and reuse them during inference. However, directly combining KV caches from multiple documents can significantly degrade generation quality because of positional inconsistencies and contextual interactions between documents. In this project, we investigate selective KV cache re-computation strategies, where only a small subset of tokens is recomputed during the prefill stage while the remaining KV representations are reused. The main objective is to discover token-level importance metrics that identify which tokens should be recomputed in order to preserve generation quality while minimizing computational cost and TTFT.

1 Introduction

Modern LLM systems often improve response quality by augmenting the user query with additional textual context. This approach is widely used in RAG systems, where relevant documents are retrieved from an external knowledge base and appended to the prompt before generation. By providing the model with task-specific or domain-specific information, RAG systems can significantly improve factual accuracy and reduce hallucinations.

However, adding large amounts of retrieved text substantially increases inference latency. Before generating any output token, an LLM performs a prefill stage, where the entire input sequence is processed token by token through the transformer model in order to construct the KV cache used during autoregressive generation. Since the computational cost of prefill scales with the length of the input context, long retrieved documents can

dramatically increase the TTFT, which is one of the most important latency metrics in interactive LLM applications.

A straightforward way to mitigate this cost is to reuse KV caches computed for document chunks. Figure 1 illustrates several strategies for KV cache reuse and their trade-offs.

Scenario 1: Full KV re-compute (default). In the standard setting, the model concatenates all retrieved chunks and recomputes the KV cache over the entire input during prefill (Figure 1a). Although this approach preserves the best generation quality, it is also the slowest option, since every token in the input must be processed from scratch before generation can begin.

Scenario 2: Prefix caching. If the beginning of the input, for example the system prompt or a shared query prefix, remains unchanged across requests, its KV cache can be stored and reused. In this case, only the remaining tokens must be processed during prefill (Figure 1b). While this reduces some computation, the speedup is usually limited because the reusable prefix is often much shorter than the retrieved document context.

Scenario 3: Full KV reuse. Another extreme is to reuse pre-computed KV caches for all retrieved chunks without performing additional computation (Figure 1c). This approach can almost eliminate prefill cost and significantly reduce TTFT. However, independently computed KV caches do not capture cross-chunk attention interactions and may contain positional inconsistencies between documents. As a result, full KV reuse often leads to noticeable degradation in generation quality.

These limitations motivate a more balanced approach: **selective KV re-computation**. Instead of recomputing the entire context, the system reuses cached KV representations for most tokens while selectively recomputing only a small subset of im-

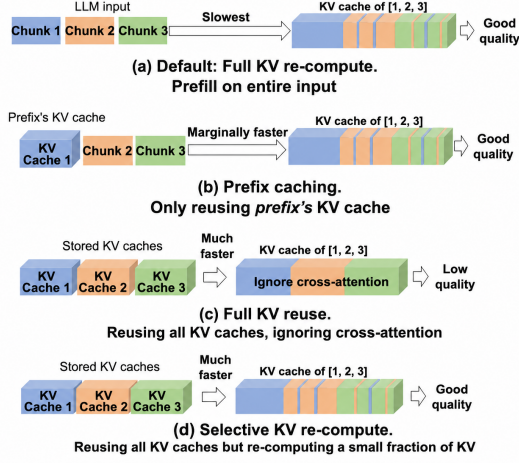


Figure 1: Comparison of different KV cache reuse strategies.

portant tokens during prefill (Figure 1d). By re-computing only these important tokens, it may be possible to maintain high generation quality while adding only a modest amount of extra computation and still achieving substantially lower TTFT compared to full recomputation.

2 Methods

Our work focuses on selective KV cache re-computation strategies for position-independent KV cache reuse in RAG systems. The main goal is to identify a small subset of tokens whose recomputation can effectively restore cross-chunk attention while preserving low prefill latency.

2.1 Difference-Based Token Selection

We implemented several token selection strategies inspired by CacheBlend (Jiayi Yao, 2025). CacheBlend proposes selectively recomputing tokens whose recomputed KV representations differ the most from their cached versions, under the intuition that these tokens are the most affected by missing cross-chunk attention.

We explored the following difference-based selection methods:

- `topk_diff_k`
- `topk_diff_v`
- `topk_diff_kv_sum`
- `topk_diff_kv_weighted`

The `topk_diff_k` strategy selects tokens with the largest discrepancy in key vectors between recomputed and cached representations,

while `topk_diff_v` uses value-vector differences. The `topk_diff_kv_sum` method combines both key and value discrepancies with equal importance, whereas `topk_diff_kv_weighted` uses a weighted combination of key and value differences.

2.2 Locality-Aware Expansion Strategies

Inspired by the group-based recomputation strategy proposed in CacheClip (Bin Yang, 2025), we additionally explored locality-aware token expansion methods. Instead of recomputing isolated top-k difference tokens, these approaches expand recomputation regions around important tokens in order to better preserve local contextual coherence.

We implemented two locality-aware strategies:

- `topk_diff_k_neighbors`
- `topk_diff_k_span_closing`

The `topk_diff_k_neighbors` strategy starts from the selected high-difference tokens and additionally recomputes nearby neighboring tokens within a fixed recomputation budget. The intuition is that partially recomputing semantically connected token groups may better preserve contextual integrity compared to isolated sparse updates.

The `topk_diff_k_span_closing` strategy expands recomputation spans by filling small gaps between selected tokens, producing more contiguous recomputation regions. This approach is motivated by the observation that fragmented recomputation can break semantic continuity, especially for multi-token entities or phrases.

2.3 Boundary Token Re-computation

Inspired by LegoLink from EPIC (Junhao Hu, 2025), which demonstrates that recomputing contiguous chunk-boundary regions can effectively mitigate attention sink artifacts, we additionally explored boundary-focused recomputation strategies.

In transformer models, independently cached chunks often exhibit repeated attention sink behavior near chunk boundaries, especially at the beginning of each chunk. Following the intuition of EPIC, recomputing boundary regions may help restore more natural attention distributions and reduce positional inconsistencies between independently computed KV caches.

2.4 Fusion Strategies

Finally, we explored several KV fusion strategies inspired by CacheSlide (Yang Liu, 2026). CacheSlide proposes weighted fusion between cached and recomputed KV representations instead of fully overwriting cached states after recomputation.

Let K_{old} , V_{old} denote the cached key and value representations, and let K_{new} , V_{new} denote the recomputed representations obtained during selective recomputation. Instead of completely replacing the cached states,

$$K_{fused} = K_{new}, \quad V_{fused} = V_{new},$$

we additionally explored weighted fusion schemes of the form

$$K_{fused} = \alpha K_{new} + (1 - \alpha) K_{old},$$

$$V_{fused} = \beta V_{new} + (1 - \beta) V_{old},$$

where $\alpha, \beta \in [0, 1]$ control the contribution of recomputed and cached representations.

The intuition behind this approach is that cached KV states may still contain partially useful information despite missing cross-chunk attention. Weighted fusion therefore attempts to preserve useful cached information while gradually incorporating updated contextual information from recomputation.

We experimented with both fixed and adaptive weighting coefficients during selective updates. In the adaptive setting, the fusion weights depended on the discrepancy between cached and recomputed KV representations, assigning larger weights to recomputed states for tokens with larger KV differences. The intuition is that tokens strongly affected by missing cross-chunk attention should rely more heavily on recomputed representations, while tokens with smaller discrepancies may still benefit from preserving their original cached states.

3 Evaluation

3.1 Experimental Setup

We evaluated the proposed selective KV recomputation strategies on the WikiMultiHopQA (2WikiMQA) benchmark, a multi-hop question answering dataset designed to evaluate reasoning across multiple documents. The dataset requires the model to combine information from several retrieved text chunks in order to answer a query,

Table 1: Quality and latency comparison between baseline recomputation and CacheBlend-based recomputation.

Method	Avg F1	Avg TTFT (s)
Baseline Full Recompute	0.350532	0.724556
CacheBlend Full Recompute	0.341969	0.922810

making it a suitable benchmark for studying cross-chunk attention and KV cache reuse in RAG systems.

All experiments were conducted using the DeepSeek-R1-Distill-Qwen-32B model. To support selective KV cache recomputation and custom KV fusion strategies, we implemented our methods as modifications to LMCache (lmc), an open-source framework for KV cache management built on top of vLLM.

Experiments were executed on multiple NVIDIA Tesla V100 GPUs. We measured both generation quality and inference efficiency in order to evaluate the trade-off between selective recomputation cost and answer accuracy.

3.2 Experiments

Figure 2(a) shows the effect of different selective recomputation ratios on answer quality. The experiments follow the CacheBlend-style selective recomputation setup. We observe that increasing the recomputation ratio beyond 16% does not consistently improve generation quality. In particular, intermediate recomputation ratios such as 24% and 32% even lead to slightly worse results. Only the 100% recomputation setting, corresponding to full KV recomputation, provides a substantial quality improvement.

Figure 2(b) illustrates the effect of weighted KV fusion while keeping the recomputation ratio fixed at 16%. The x-axis shows the percentage of reused cached KV representations preserved during fusion. We observe that relying too heavily on old cached KV states tends to reduce generation quality, suggesting that stale cached representations may preserve positional inconsistencies and missing cross-chunk attention patterns. At the same time, moderate reuse levels achieve comparable quality while still preserving part of the computational benefits of KV reuse.

Table 1 compares the standard full recomputation baseline¹ with the CacheBlend-based recomputation pipeline using a 100% recomputation ratio. Interestingly, despite recomputing all tokens, the

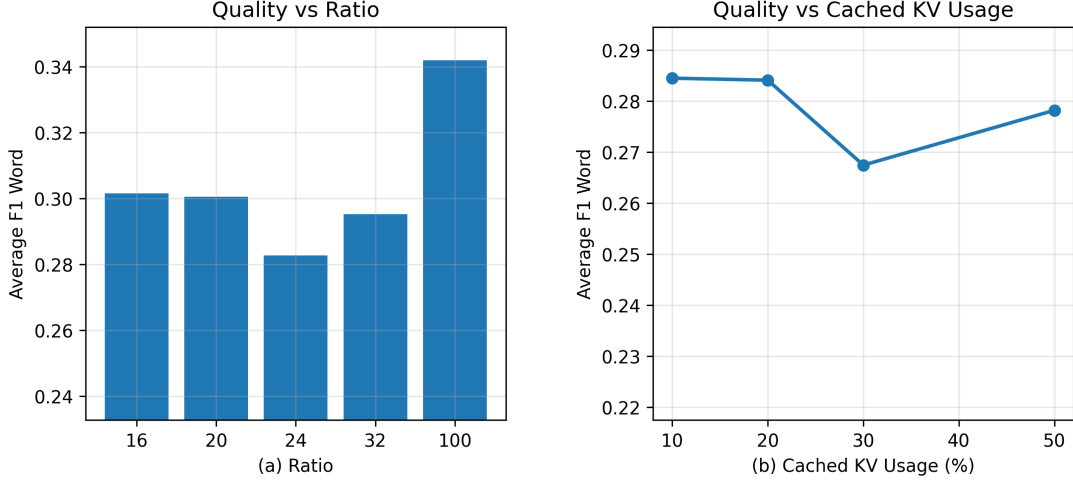


Figure 2: Quality comparison for different selective recomputation settings. (a) CacheBlend-style selective recomputation with different recomputation ratios. (b) Weighted KV fusion with different cached KV usage percentages while keeping the recomputation ratio fixed at 16%.

CacheBlend implementation still achieves slightly worse quality compared to the baseline full recomputation setup.

A possible explanation is that the LMCACHE selective recomputation pipeline relies on its own custom transformer implementation for KV comparison and recomputation, whereas the baseline directly uses the original model implementation during prefill. Small numerical or implementation-level differences between these computation paths may accumulate and lead to observable quality degradation, even under full recomputation.

In addition, the CacheBlend-based pipeline introduces higher TTFT compared to the baseline. This behavior is expected, since selective recomputation requires multiple additional operations, including KV cache loading, KV comparison, token selection logic, and fusion-related computations.

Table 2 presents the effect of different weighting schemes for K-difference and V-difference during token selection. The results indicate that K-difference is a more reliable signal for selecting tokens for recomputation in our setup. The highest performance is achieved when only K-difference is used, while adding a small contribution from V-difference (0.7/0.3) leads to only a minor degradation. However, configurations that rely more heavily on V-difference do not improve quality, and the V-only setup produces the lowest Average F1 Word score. Equal weighting of K- and V-differences also performs worse than the K-focused variants,

Table 2: Impact of different weighting coefficients for K-difference and V-difference during token selection for selective recomputation. Relative F1 is normalized with respect to the best configuration.

K Weight	V Weight	Average F1 Word	Relative F1 (%)
1.0	0.0	0.301610	100.00
0.7	0.3	0.300010	99.47
0.3	0.7	0.299344	99.25
0.0	1.0	0.289536	95.99
1.0	1.0	0.294123	97.52

Table 3: Impact of neighbor expansion settings in the topk_diff_k_neighbors token selection strategy. Relative F1 is normalized with respect to the best configuration.

Expand Top Ratio	Final Recompute Ratio	Average F1 Word	Relative F1 (%)
0.25	0.24	0.293938	99.75
0.25	0.32	0.294026	99.78
0.50	0.24	0.288095	97.77
0.50	0.32	0.288064	97.76
1.00	0.48	0.294669	100.00

suggesting that V-difference does not provide a clear benefit for this selection strategy.

Table 3 shows the impact of different neighbor expansion settings for the topk_diff_k_neighbors strategy. The best result is obtained when neighbor expansion is applied to all selected top-k tokens with a larger final recomputation ratio. At the same time, moderate expansion settings with smaller recomputation budgets achieve very similar quality.

Similarly, Table 4 reports the results for the topk_diff_k_span_closing strategy with different span closing parameters. The best performance is achieved with a smaller maximum gap value,

¹Standard vLLM inference without CacheBlend functionality.

Table 4: Impact of span closing settings in the topk_diff_k_span_closing token selection strategy. Relative F1 is normalized with respect to the best configuration.

Max Gap	Final Recompute Ratio	Average F1 Word	Relative F1 (%)
1	0.24	0.297113	100.00
2	0.24	0.290034	97.62
1	0.32	0.293049	98.63
2	0.32	0.290034	97.62

while increasing the allowed gap slightly reduces quality. At the same time, changing the final re-computation ratio does not noticeably affect the results for the evaluated configurations. Overall, the differences between the tested settings remain relatively small.

Overall, the evaluated modifications did not produce substantial improvements over the CacheBlend-style selective recomputation approach. Experiments with boundary token recomputation and adaptive fusion weights also showed results that were largely comparable to the baseline configuration.

In general, the best results were obtained using the standard CacheBlend-style setup with token selection based on K-difference and complete replacement of cached KV states by recomputed representations during fusion. Most alternative strategies produced either very similar or slightly worse results while introducing additional complexity into the recomputation pipeline.

4 Conclusion

In this work, we explored several selective KV cache recomputation strategies for efficient multi-document KV cache reuse in RAG systems. We investigated different token selection methods, locality-aware recomputation strategies, boundary token recomputation, and weighted KV fusion approaches inspired by recent KV cache reuse systems. Experimental results on the 2WikiMQA benchmark showed that most evaluated modifications produced results similar to or slightly worse than the CacheBlend-style approach based on K-difference token selection and full KV replacement during fusion. Overall, the experiments show that the evaluated modifications did not provide substantial improvements over the original CacheBlend-style approach.

References

- Lmcache: Efficient kv cache management for llm serving. <https://github.com/LMCache/LMCache>. Accessed: 2026-05-09.
- Jun Zeng Zhenhua Wu Bin Yang, Qiuyu Leng. 2025. *Cacheclip: Accelerating long-context llm inference via token-level kv cache compression*. *arXiv preprint arXiv:2510.10129*.
- Yuhan Liu Siddhant Ray Yihua Cheng Qizheng Zhang Kuntai Du Shan Lu Junchen Jiang Jiayi Yao, Hanchen Li. 2025. *Cacheblend: Fast large language model serving for rag with cached knowledge fusion*. *arXiv preprint arXiv:2405.16444*.
- Weidong Wang Haoyi Wang Tiancheng Hu Qin Zhang Hao Feng Xusheng Chen Yizhou Shan Tao Xie Junhao Hu, Wenrui Huang. 2025. *Epic: Efficient position-independent context caching for serving large language models*. *arXiv preprint arXiv:2410.15332*.
- Liqiang Zhang Chentao Wu Guangtao Xue Jie Li Minyi Guo Junhao Hu Jie Meng Yang Liu, Yunfei Gu. 2026. *Cacheslide: Unlocking cross position-aware kv cache reuse for accelerating llm serving*. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST '26)*, pages 83–99, Santa Clara, CA, USA.